# Software Design Specification

for

# Career & Lifelong Learning Pathway Application

Cal Poly Humboldt – Software Engineering – CS 458

Prepared by Roselie Freixas

**Contributors:** Clay Cordary, Roselie Freixas, Jerry Huang, Skyler Hunsinger

**Version:** 1.0 WIP

**Date Created:** September 13, 2026

**Date Last Modified:** October 14, 2026

# Table of Contents

**Table of Contents** ..... **2**
**1. Introduction** ..... **4**
1.1 Purpose and Intended Audience ..... 4
1.2 Document Scope ..... 4
1.3 References ..... 4
**2. System Architecture** ..... **5**
2.1 Architectural Overview ..... 5
2.2 Application Architecture Diagram ..... 6
2.3 Infrastructure Architecture Diagram ..... 7
2.4 Frontend Architecture ..... 8
2.5 Backend Architecture ..... 8
2.6 Database Architecture ..... 8
2.7 Integration Architecture ..... 9
**3. Data Design** ..... **10**
3.1 Data Model ..... 10
3.1.1 Base Entity Tables ..... 11
3.1.2 Multi-Value Handler Junction Tables for Normalization ..... 12
3.2 Relational Schema ..... 12
3.2.1 Base Entities: ..... 12
3.2.2 Multi-Value Entities: ..... 13
3.3 Entity-Relationship Diagram ..... 14
3.4 Data Dictionary ..... 15
3.4.1 career_resource ..... 15
3.4.2 category ..... 16
3.4.3 filter ..... 16
3.4.4 source ..... 16
3.4.5 curated_collection ..... 17
3.4.6 account ..... 17
3.4.7 price_model ..... 17
3.4.8 collection_resources ..... 18
3.4.9 resource_category ..... 18
3.4.10 resource_filter ..... 18
3.4.11 user_interests ..... 18
3.4.12 category_parent ..... 19
**4. Interface Design** ..... **20**
4.1 User Interface Design ..... 20
4.2 Frontend-Backend Interface Design ..... 20

4.3 External Service Interfaces..... 20
4.4 API Design..... 21
4.5 Authentication and Account-Linking Interfaces..... 21
**5. Functional Area Design..... 22**
5.1 Non-User Specific Landing Page..... 22
5.1.1 Components..... 22
5.1.2 Data..... 22
5.1.3 Interfaces..... 22
5.1.4 Behavior and Interaction Flow..... 23
5.1.5 Test Plan..... 23
5.2 Resource Discovery with Filtering..... 25
5.2.1 Components..... 25
5.2.2 Data..... 25
5.2.3 Interfaces..... 25
5.2.4 Behavior and Interaction Flow..... 26
5.2.5 Test Plan..... 26
5.3 Onboarding: User Career and Skill Profiling..... 27
5.3.1 Components..... 27
5.3.2 Data..... 28
5.3.3 Interfaces..... 28
5.3.4 Behavior and Interaction Flow..... 28
5.3.5 Test Plan..... 29
**6. Security and Privacy Design..... 31**
6.1 Application Authentication and Authorization..... 31
6.2 External Account Credential and Token Security..... 31
6.3 Data Protection and Privacy..... 31
6.4 Input Validation and Application Security..... 31
**7. Error Handling and Reliability..... 32**
7.1 Error Handling Strategy..... 32
7.2 External Service Failure Handling..... 32
7.3 Data Integrity and Recovery..... 32
7.4 Logging and Diagnostics..... 32
**8. Design Rationale..... 33**
8.1 Major Design Decisions..... 33
8.2 Alternatives Considered..... 33
8.3 Tradeoffs and Limitations..... 33
**Appendix A: Glossary..... 35**
**Appendix B: AI Assistance Disclosure..... 36**

# 1. Introduction

## 1.1 Purpose and Intended Audience

The purpose of this document is to detail the projected design specifications for the proposed *Career & Lifelong Learning Pathway (CaLLP)* application. This document is intended for developers, program managers, and other interested parties who wish to ascertain the technical infrastructure and architecture of the CaLLP application.

## 1.2 Document Scope

This document contains specifications for project architecture design, infrastructure design, data design, interface design, and other technical design details which support the features and functional areas outlined in the CaLLP Software Requirements Specification (SRS) document. This document is intended to be a companion to the SRS document.

## 1.3 References

The following developer resources should be referenced while building and maintaining The *CaLLP* application:

- CaLLP Software Requirements Specification document
- CaLLP GitHub Organization:
  - https://github.com/cph-career-learning-pathways
- CaLLP GitHub Repository:
  - https://github.com/cph-career-learning-pathways/callp-app
- Accessibility Best Practices for Web Development:
  - https://www.w3.org/TR/2024/REC-WCAG22-20241212/
  - https://developer.mozilla.org/en-US/docs/Web/Accessibility
- Account Linking Best Practices for Cross-platform Authentication:
  - https://www.loginradius.com/blog/identity/account-linking-social-login-ux
- Cal Poly Humboldt Single Sign On (SSO) Information:
  - https://www.humboldt.edu/its/saml-sso-request-form
- Let's Encrypt Subscriber Agreement (July-2026) for HTTPS TLS Certificates:
  - https://letsencrypt.org/documents/LE-SA-v1.8-July-06-2026.pdf

# 2. System Architecture

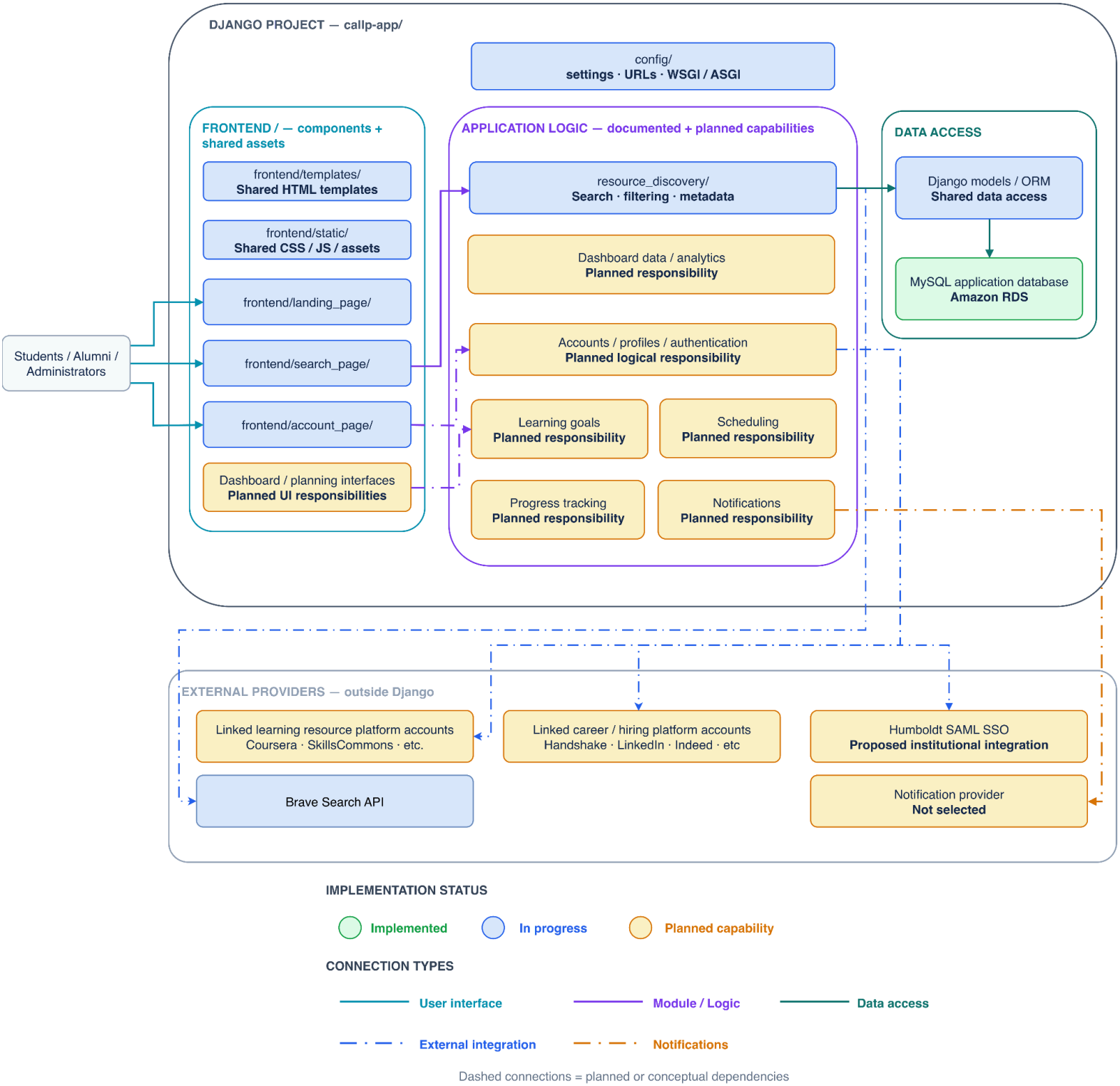
## 2.1 Architectural Overview

The CaLLP application utilizes Django, a Python web framework for developing database-driven web applications. The application follows a layered architecture, separating the user interface, application logic, and data storage responsibilities. The frontend provides user interaction and presentation, while the Django backend manages application logic, authentication, data processing, and database operations. Amazon Relational Database Service (RDS), utilizing MySQL, provides persistent storage for application data, including user information, learning goals, resources, and progress tracking.

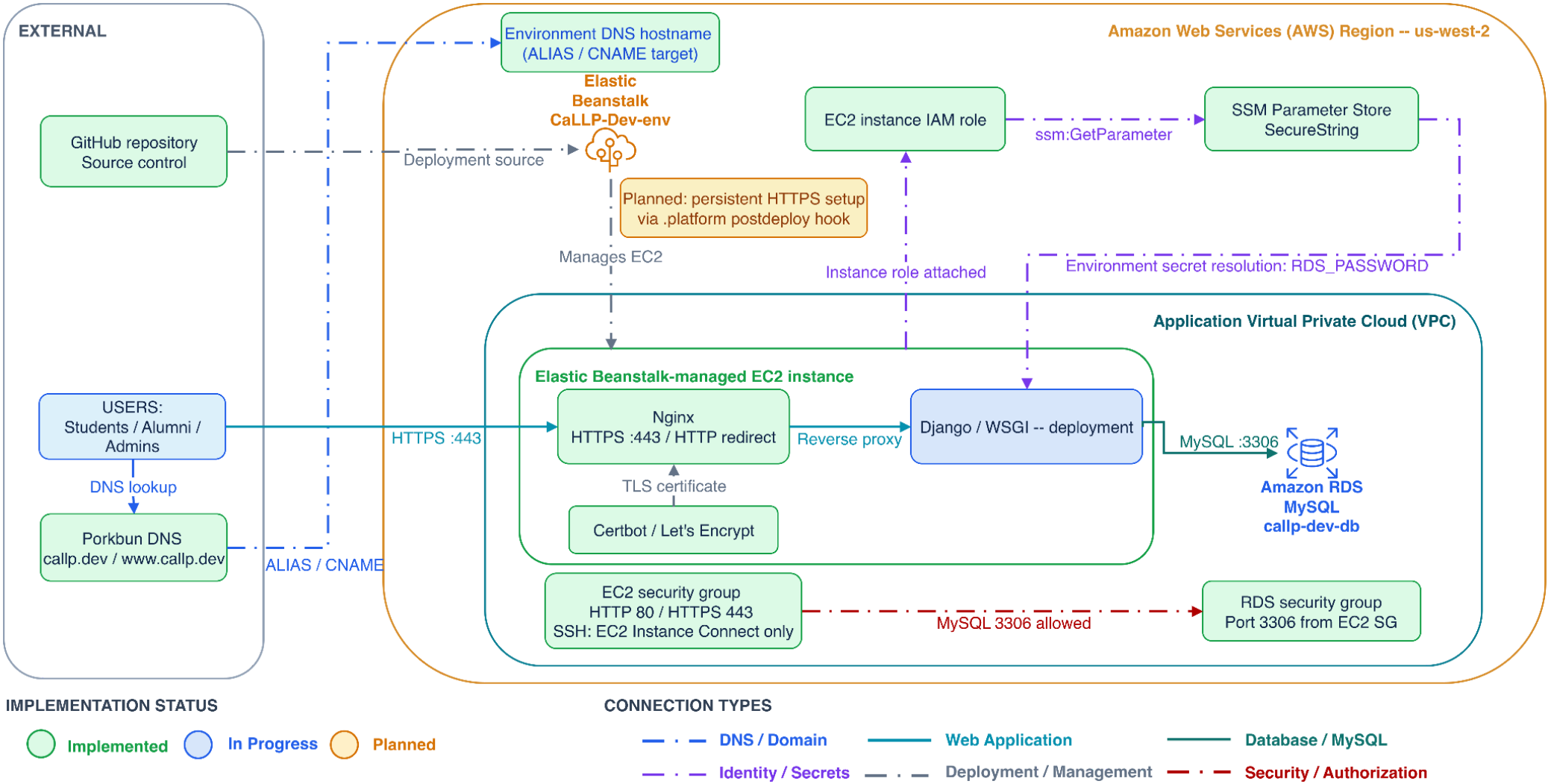
The CaLLP application serves as an intermediary between users and external learning and career development platforms rather than hosting learning content directly. External integrations currently include the Brave Search API for learning resource discovery. Planned integrations include Cal Poly Humboldt Single Sign-On (SSO) for authentication, learning resource and career platform account integrations, and a third-party notification provider API (provider to be determined). Please refer to **Section 2.2 Application Architecture Diagram** for a visual depiction of the current and planned application architecture.

The CaLLP application utilizes Amazon Web Services (AWS) for application hosting and database infrastructure. AWS Elastic Beanstalk provides the environment for deploying and running the Django web application, while Amazon RDS provides managed MySQL database hosting. The application is accessible through the domain callp.dev, registered via Porkbun domain name registrar, with www.callp.dev also supported. Domain Name System (DNS) configuration directs incoming traffic to the deployed application, while HyperText Transfer Protocol Secure (HTTPS) provides encrypted communication between users and the web server. Additional AWS services support networking, security, and application configuration. Please refer to **Section 2.3 Infrastructure Architecture Diagram** for a visual depiction of the current and planned infrastructure architecture.

## 2.2 Application Architecture Diagram



## 2.3 Infrastructure Architecture Diagram



## 2.4 Frontend Architecture

The CaLLP application utilizes HTML, CSS, and JavaScript for its web-based user interface. Django's template system is used to generate dynamic pages and reuse common interface elements throughout the application. Following the modular monolithic architecture, the frontend is organized around application features such as user authentication, resource discovery, learning goals, progress tracking, and administrative functions.

Django handles page routing and communication between the frontend and backend. User requests are processed through Django views, which retrieve or update application data and return the appropriate responses. JavaScript is used for additional frontend functionality and user interaction where needed.

## 2.5 Backend Architecture

The CaLLP application utilizes Django's Model-View-Template (MVT) architecture for backend development. Django models define the application's data structures and interact with the MySQL database through Django's Object-Relational Mapping (ORM). Views handle incoming requests, application logic, and communication between the frontend, database, and external services.

Following the modular monolithic architecture, backend functionality is organized into Django applications and modules based on their responsibilities. These include user management, resource discovery, learning goal management, progress tracking, and administrative functions. The backend also handles external API integrations, including the Brave Search API and planned integrations with learning and career platforms. Sensitive information, such as API credentials, is managed server-side rather than exposed to the frontend.

## 2.6 Database Architecture

The CaLLP application utilizes MySQL as its database, hosted through AWS RDS. The database stores application data including user information, learning resources, learning goals, and progress tracking. Django communicates with MySQL through its ORM to retrieve, store, and update data. Django migrations are used to manage changes to the database structure.

AWS RDS provides database hosting, while Elastic Beanstalk hosts the Django application. The two services communicate through AWS networking and security configurations. Hosting through AWS will also support the eventual transfer of the application to its new owner, with source code and documentation provided for continued development and maintenance.

Please refer to **Section 3 Data Design** for detailed database tables, fields, keys, and relationships.

## 2.7 Integration Architecture

The CaLLP application utilizes external services for resource discovery, user authentication, and integration with learning and career platforms. External services communicate with the Django backend through APIs and supported authentication protocols.

The Brave Search API is currently used for learning resource discovery. Its API key is stored as an environment variable, loaded through the application's virtual environment configuration, rather than directly in the source code. Planned learning and career platform integrations will utilize OAuth 2.0 with Proof Key for Code Exchange (PKCE) for secure account authorization. Cal Poly Humboldt SSO integration is planned to utilize Security Assertion Markup Language (SAML) for user authentication.

The CaLLP application is hosted through AWS Elastic Beanstalk, with additional AWS services supporting database access, networking, security, and configuration. External service credentials and configuration settings will be managed server-side to prevent exposure to users.

# 3. Data Design

The current data design currently accommodates the following CaLLP application functional areas: **Non-User Specific Landing Page** (**FR-3.13.2**), **Learning Resource Discovery with Filtering** (**FR-3.4.2**), and **User Career and Skill Profiling** (**FR-3.2.1**). It also partially accommodates **Administrative Actions**.

## 3.1 Data Model

The CaLLP database utilizes relational tables to store application data, including users, learning resources, categories, and user interests. Multi-value tables are used to establish relationships between the different entities.

**Base Tables**

- **career_resource:** Contains information about each resource, such as its URL, name, who added it to the database, and a relevant snippet. It also includes information such as resource type, format, duration, and pricing.
- **category:** Contains category names so that resources and user interests can be assigned to them through relevant multi-value tables. Categories include skills, degree programs, and career fields.
- **source:** Contains websites and platforms curated by developers and admin users. Sources can be classified as trusted, normal, or blocked.
- **filter:** Contains the names of relevant filter options selectable in user searches, such as difficulty, pacing, credentials, and accessibility.
- **account:** Contains relevant user data for identification, contact, account roles, and status.
- **curated_collection:** Contains collections of resources, allowing users at certain permission levels to curate collections of resources.
- **price_model:** Contains pricing classifications that determine whether a resource is free, freemium, subscription-based, one-time payment, etc.

**Multi-Value Tables**

- **collection_resources:** Connects a resource to a curated collection and identifies its order within the collection.
- **resource_category:** Assigns categories to resources, allowing a resource to be associated with multiple categories.
- **resource_filter:** Assigns filters to resources, allowing resources to be associated with multiple search filters.

- **user_interests:** Connects users to categories they are interested in, such as skills, degree programs, and career fields. It also allows users to assign an interest score to skills.
- **category_parent:** Connects skills to relevant degree programs and career fields, allowing relationships between these categories to be established.

### 3.1.1 Base Entity Tables

| **Career_Resource** ✅ | **Category** ✅ | **Filter** ✅ |
|---|---|---|
| RESOURCE_ID (PK)<br>curator_id (FK)<br>is_anonymous<br>source_id (FK)<br>resource_title<br>resource_url<br>resource_type<br>resource_format<br>resource_snippet<br>resource_pub_date<br>resource_refresh_date<br>resource_duration<br>price_model_id (FK NULL)<br>other_metadata (JSON NULL)<br>is_active | CATEGORY_ID (PK)<br>curator_id (FK)<br>category_type<br>category_name<br>is_active | FILTER_ID (PK)<br>curator_id (FK)<br>filter_type<br>filter_name<br>is_active |
| **Source** ✅ | **Curated_Collection** ✅ | **Account** ✅ |
| SOURCE_ID (PK)<br>curator_id (FK)<br>source_name<br>source_domain<br>source_trust<br>source_add_date<br>source_up_date<br>is_active | COLLECTION_ID (PK)<br>curator_id (FK)<br>collection_name<br>collection_description<br>collection_add_date<br>collection_up_date<br>is_active | acc_ID (PK)<br>acc_fname<br>acc_lname<br>acc_email<br>acc_role<br>acc_status |

| **Price_Model** ✅ | | |
|---|---|---|
| PRICE_MODEL_ID (PK)<br>price_model_name<br>is_active | | |

### 3.1.2 Multi-Value Handler Junction Tables for Normalization

| **Resource_Category** ✅ | **Resource_Filter** ✅ | **User_Interests** ✅ |
|---|---|---|
| RESOURCE_ID (PK/FK)<br>CATEGORY_ID (PK/FK) | RESOURCE_ID (PK/FK)<br>FILTER_ID (PK/FK) | ACC_ID (PK/FK)<br>CATEGORY_ID (PK/FK)<br>intr_date<br>intr_score |
| **Category_Parent** ✅ | **Collection_Resources** ✅ | |
| SKILL_ID(PK/FK)<br>PARENT_ID(PK/FK) | collection_id (PK/FK)<br>resource_id (PK/FK)<br>sort_order<br>add_date | |

## 3.2 Relational Schema

### 3.2.1 Base Entities:

**career_resource** (RESOURCE_ID, curator_id, is_anonymous, source_id, resource_title, resource_url, resource_type, resource_format, resource_snippet, resource_pub_date, resource_refresh_date, resource_duration, price_model_id, other_metadata, is_active)
foreign key (curator_id) references account
foreign key (source_id) references source
foreign key (price_model_id) references price_model

**category** (CATEGORY_ID, curator_id, category_type, category_name, is_active)
foreign key (curator_id) references account

**filter** (FILTER_ID, curator_id, filter_type, filter_name, is_active)
foreign key (curator_id) references account

**source** (SOURCE_ID, curator_id, source_name, source_domain, source_trust, source_add_date, source_up_date, is_active)
foreign key (curator_id) references account

**curated_collection** (COLLECTION_ID, curator_id, collection_name, collection_description, collection_add_date, collection_up_date, is_active)
foreign key (curator_id) references account

**account** (ACC_ID, acc_fname, acc_lname, acc_email, acc_role, acc_status)

**price_model** (PRICE_MODEL_ID, price_model_name, is_active)

### 3.2.2 Multi-Value Entities:

**resource_category** (RESOURCE_ID, CATEGORY_ID)
foreign key (resource_id) references career_resource
foreign key (category_id) references category

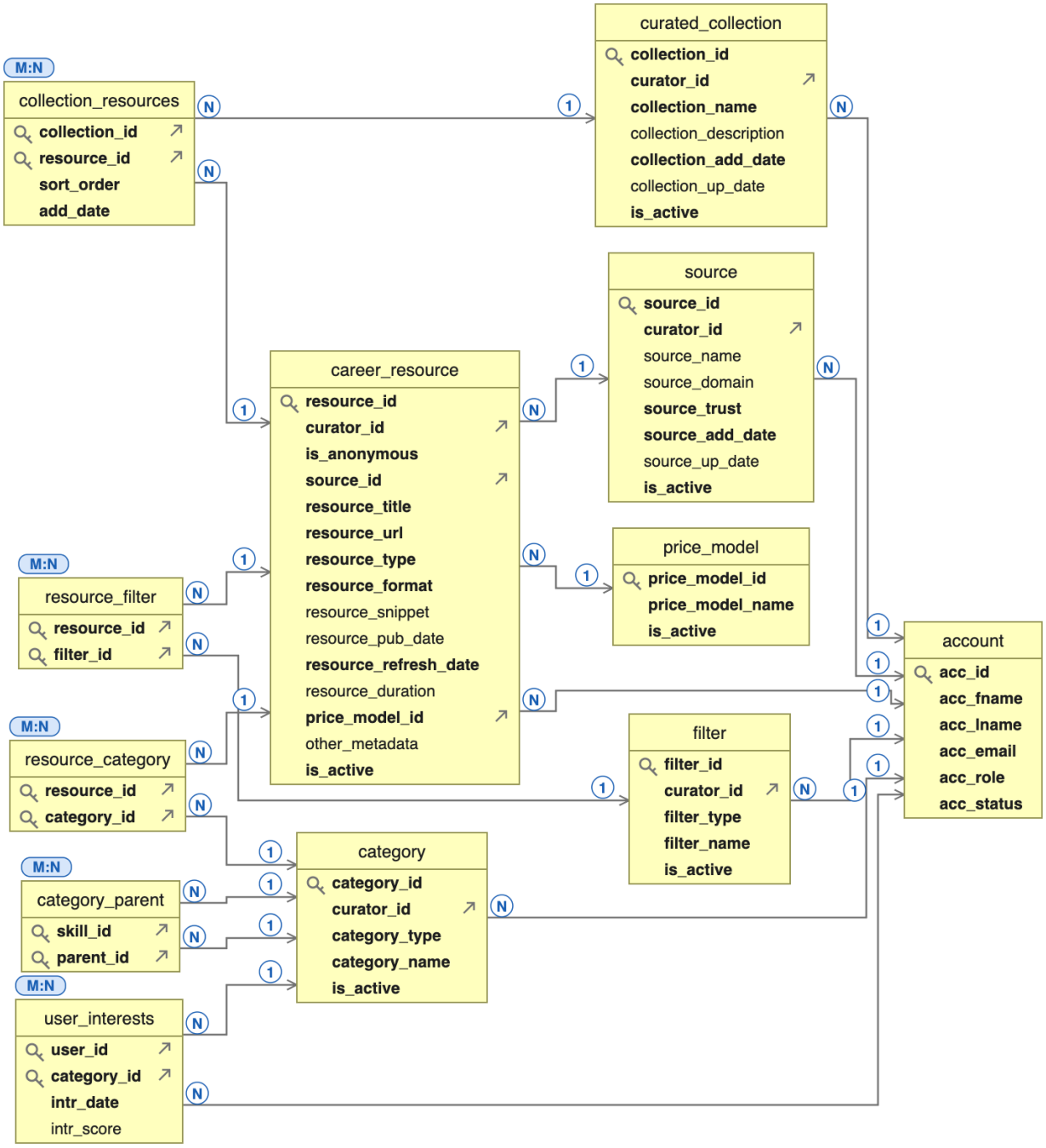
**collection_resources** (COLLECTION_ID, RESOURCE_ID, sort_order, add_date)
foreign key (collection_id) references curated_collection
foreign key (resource_id) references career_resource

**resource_filter** (RESOURCE_ID, FILTER_ID)
foreign key (resource_id) references career_resource
foreign key (filter_id) references filter

**user_interests** (USER_ID, CATEGORY_ID, intr_date, intr_score)
foreign key (user_id) references account
foreign key (category_id) references category

**category_parent** (SKILL_ID, PARENT_ID)
foreign key (skill_id) references category
foreign key (parent_id) references category

## 3.3 Entity-Relationship Diagram




**Key**
1 / N at the ends of a line = one-to-many (1 on the referenced table, N on the table holding the foreign key).
M:N tag = junction table, many-to-many between the two tables it links:
collection_resources: curated_collection to career_resource resource_category: career_resource to category
resource_filter: career_resource to filter category_parent: category to category (self) user_interests: account to category
1:1 = none in this schema (no foreign key is unique).

## 3.4 Data Dictionary

The following tables list the attributes, data types, and constraints in the currently implemented MySQL schema. Planned attributes not included in the current sql schema are excluded:

### 3.4.1 career_resource

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| resource_id | char(8) | unique 8-digit identifier for each career resource | primary key; not null; check (resource_id regexp '^[0-9]{8}$') |
| curator_id | char(8) | user who created the resource record | foreign key → account.acc_id; not null; default '11111111' |
| is_anonymous | char(1) | whether the curator is hidden from public view | not null; default 'N'; check (is_anonymous in ('Y', 'N')) |
| source_id | char(8) | source associated with the resource | foreign key → source.source_id; not null |
| resource_title | varchar(150) | title displayed to users | not null |
| resource_url | varchar(767) | unique url linking to the external resource | not null |
| resource_type | varchar(15) | type of resource | not null; check (resource_type in ('course', 'article', 'collection')) |
| resource_format | varchar(20) | format of resource content | not null; check (resource_format in ('video', 'document', 'webpage', 'mixed')) |
| resource_snippet | text | short description for resource previews | nullable |
| resource_pub_date | datetime | original publication date, when known | nullable |
| resource_refresh_date | datetime | date and time resource metadata was last refreshed | not null; default (now()) |
| resource_duration | bigint unsigned | estimated duration in seconds, when known | default null |
| price_model_id | char(8) | pricing model assigned to the resource | foreign key → price_model.price_model_id; not null |
| other_metadata | json | additional resource-specific metadata | default null |
| is_active | char(1) | whether the resource is currently in use | not null; default 'Y'; check (is_active in ('Y', 'N')) |

table constraint: unique (resource_url).

### 3.4.2 category

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| category_id | char(8) | unique 8-digit identifier for each category | primary key; not null; check (category_id regexp '^[0-9]{8}$') |
| curator_id | char(8) | user responsible for creating and maintaining the category | foreign key → account.acc_id; not null; default '11111111' |
| category_type | varchar(20) | type of category | not null; check (category_type in ('skill', 'degree program', 'career field')) |
| category_name | varchar(100) | display name of the category | not null |
| is_active | char(1) | whether the category is currently in use | not null; default 'Y'; check (is_active in ('Y', 'N')) |

table constraint: unique (category_type, category_name).

### 3.4.3 filter

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| filter_id | char(8) | unique 8-digit identifier for each filter | primary key; not null; check (filter_id regexp '^[0-9]{8}$') |
| curator_id | char(8) | user responsible for creating and maintaining the filter | foreign key → account.acc_id; not null; default '11111111' |
| filter_type | varchar(30) | grouping of related filters | not null |
| filter_name | varchar(30) | name of the filter used in searches | not null |
| is_active | char(1) | whether the filter is currently in use | not null; default 'Y'; check (is_active in ('Y', 'N')) |

table constraint: unique (filter_type, filter_name).

### 3.4.4 source

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| source_id | char(8) | unique 8-digit identifier for each source | primary key; not null; check (source_id regexp '^[0-9]{8}$') |
| curator_id | char(8) | user who provided the source | foreign key → account.acc_id; not null; default '11111111' |
| source_name | varchar(50) | name of the source | nullable |
| source_domain | varchar(253) | url or domain name of the source | nullable |
| source_trust | varchar(7) | trust classification of the source | not null; default 'normal'; check (source_trust in ('trusted', 'normal', 'blocked')) |
| source_add_date | datetime | date and time the source was added | not null; default (now()) |
| source_up_date | datetime | date and time the source was last checked or vetted | default null |
| is_active | char(1) | whether the source is currently active | not null; default 'Y'; check (is_active in ('Y', 'N')) |

### 3.4.5 curated_collection

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| collection_id | char(8) | unique 8-digit identifier for each collection | primary key; not null; check (collection_id regexp '^[0-9]{8}$') |
| curator_id | char(8) | user who created the collection | foreign key → account.acc_id; not null; default '11111111' |
| collection_name | varchar(100) | title of the collection | not null |
| collection_description | text | custom description of the collection | nullable |
| collection_add_date | datetime | date and time the collection was created | not null; default (now()) |
| collection_up_date | datetime | date and time the collection was last checked or updated | nullable |
| is_active | char(1) | whether the collection is currently in use | not null; default 'Y'; check (is_active in ('Y', 'N')) |

### 3.4.6 account

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| acc_id | char(8) | unique 8-digit identifier for each user | primary key; not null; check (acc_id regexp '^[0-9]{8}$') |
| acc_fname | varchar(100) | user's first name | not null |
| acc_lname | varchar(100) | user's last name | not null |
| acc_email | varchar(50) | user's email address | not null; unique; check (regexp_like(acc_email, '^[^@.]+@[^@.]+[.][^@.]+$')) |
| acc_role | varchar(10) | role of the user within the system | not null; check (acc_role in ('student', 'teacher', 'admin', 'alumni', 'developer', 'system')) |
| acc_status | varchar(20) | current status of the user account | not null; check (acc_status in ('active', 'inactive', 'suspended', 'pending', 'locked', 'archived', 'deleted')) |

table constraint: unique (acc_email).

### 3.4.7 price_model

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| price_model_id | char(8) | unique 8-digit identifier for each pricing model | primary key; not null; check (price_model_id regexp '^[0-9]{8}$') |
| price_model_name | varchar(20) | name of the pricing model | not null |
| is_active | char(1) | whether the pricing model is active or retired | not null; default 'Y'; check (is_active in ('Y', 'N')) |

table constraint: unique (price_model_name).

### 3.4.8 collection_resources

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| collection_id | char(8) | identifier of the curated collection | primary key; foreign key → curated_collection.collection_id; not null |
| resource_id | char(8) | identifier of the career resource | primary key; foreign key → career_resource.resource_id; not null |
| sort_order | tinyint unsigned | position of the resource chosen by the curator | not null; check (sort_order between 0 and 50) |
| add_date | datetime | date and time the resource was added to the collection | not null; default (now()) |

The composite primary key is (collection_id, resource_id).

### 3.4.9 resource_category

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| resource_id | char(8) | identifier of the career resource | primary key; foreign key → career_resource.resource_id; not null |
| category_id | char(8) | identifier of the associated category | primary key; foreign key → category.category_id; not null |

The composite primary key is (resource_id, category_id).

### 3.4.10 resource_filter

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| resource_id | char(8) | identifier of the career resource | primary key; foreign key → career_resource.resource_id; not null |
| filter_id | char(8) | identifier of the associated filter | primary key; foreign key → filter.filter_id; not null |

The composite primary key is (resource_id, filter_id).

### 3.4.11 user_interests

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| user_id | char(8) | identifier of the user | primary key; foreign key → account.acc_id; not null |
| category_id | char(8) | identifier of the category of interest | primary key; foreign key → category.category_id; not null |
| intr_date | datetime | date the user added the category as an interest | not null; default (now()) |
| intr_score | tinyint unsigned | user-reported skill interest score | default null; check (intr_score between 1 and 5) |

The composite primary key is (user_id, category_id).

### 3.4.12 category_parent

| Attribute | Data Type | Description | Constraints and Values |
|---|---|---|---|
| skill_id | char(8) | identifier of the skill category | primary key; foreign key → category.category_id; not null |
| parent_id | char(8) | identifier of the related degree program or career field category | primary key; foreign key → category.category_id; not null |

table constraints: skill_id must differ from parent_id. Triggers require skill_id to reference a skill and parent_id to reference a degree program or career field.

# 4. Interface Design

## 4.1 User Interface Design

The CaLLP application utilizes a responsive web interface designed for use on desktop and mobile devices. The interface will provide navigation between major application features, including resource discovery, learning goals, scheduling, progress tracking, and user account management. Administrative users will have access to additional management and reporting functions (**FR-3.11.3**).

Common layouts, navigation elements, and interface components will be reused throughout the application to maintain consistency. The design will prioritize ease of use, accessibility, and minimizing unnecessary steps for users. Accessibility considerations will include keyboard navigation, readable text, appropriate color contrast, and compatibility with assistive technologies.

Detailed interface designs and interactions for individual features are covered in **Section 5**.

## 4.2 Frontend-Backend Interface Design

The CaLLP application's frontend and Django backend communicate through HTTP/HTTPS requests. Django handles URL routing, processes requests, retrieves or updates database information, and returns the appropriate responses to the frontend. Django templates will be used to generate dynamic pages, while JavaScript may be used for additional client-side interactions and asynchronous requests.

The backend is responsible for application logic, database operations, and validation of submitted data. User sessions will be managed server-side, with error responses provided when requests fail or invalid data is submitted.

## 4.3 External Service Interfaces

The CaLLP application communicates with external services for resource discovery, authentication, and planned learning and career platform integrations.

The Brave Search API is used to retrieve external learning resources based on user searches. Requests are sent from the Django backend, and returned results are processed for use within the application (**FR-3.4.3**). API credentials are stored server-side using environment variables.

Planned integrations include Cal Poly Humboldt SSO using SAML for user authentication and OAuth 2.0 with PKCE for connecting learning and career platform accounts (**FR-3.1.1**). These integrations will allow users to authenticate or authorize access to supported external accounts without providing their credentials directly to CaLLP. A third-party notification service is also planned (provider to be determined).

External integrations will depend on the capabilities, permissions, and usage limits provided by each service.

## 4.4 API Design

The CaLLP application currently utilizes the Brave Search API for resource discovery. API requests are handled through the backend, which processes returned data before making it available to the frontend. API keys and other sensitive credentials are not exposed to the client.

Additional API integrations and internal endpoints will be documented as they are developed. This will include request methods, parameters, response formats, error handling, and applicable rate limits.

## 4.5 Authentication and Account-Linking Interfaces

The CaLLP application will support user authentication through Cal Poly Humboldt SSO using SAML (**FR-3.1.1**). Users will be directed to the university's authentication service to sign in, with authentication results returned to CaLLP to establish the user's application session. Login and logout functionality will be provided through the application's user interface.

Planned learning and career platform integrations will utilize OAuth 2.0 with PKCE for account linking. Users will be redirected to the external platform to authorize access, after which CaLLP will receive the authorization response and complete the account-linking process. Users will be able to manage linked accounts and revoke access where supported.

Authentication and account-linking failures will provide appropriate feedback to users. Security requirements for authentication, session management, and credential protection are covered in **Section 6**.

# 5. Functional Area Design

This section contains the implementation design for major functional areas identified in the CaLLP SRS. Unlike Sections 2–4, which describe system-wide design, Section 5 explains how those architectural pieces come together to implement functional feature sets.

## 5.1 Non-User Specific Landing Page

When a user visits the website for the first time, they will be greeted with an unintimidating landing page presenting a curated selection of the most generally useful collections for college students from the database (**FR-3.13.1**). When the user moves away from the page, a prompt will ask them to sign in or complete the onboarding process and sign up.

### 5.1.1 Components

The project technology stack required to make this functional area work includes Django for the frontend and database connection, MySQL for database management, and AWS as the hosting platform. Django will handle retrieving and displaying curated collections and their associated resources.

### 5.1.2 Data

The landing page reads a selection of the `curated_collection`, `collection_resources`, and `career_resource` tables to display information on the page. The source, category, and filter tables provide additional information used for displaying and filtering resources. The user can select any portion of a collection to gain further information, such as resource links, descriptions, and media formats.

For users with existing accounts, the `user_interests` table can also be used to display collections based on their interests.

### 5.1.3 Interfaces

The user interface will show stacked cards of resources that are initially curated by administrators (**FR-3.13.5**), but as usage expands could show curation from other users, including students, teachers, and alumni. Selecting any of these stacks will expand into a sidebar that lists what the stack contains, as well as tidbits about each resource should the user have any interest in exploring further (**FR-3.13.3**).

As the user makes selections, the frontend will communicate with the Django backend to retrieve the relevant resources from the database (**FR-3.13.2**). Resource links will direct users to the external platforms where the content is hosted (**FR-3.13.4**). Developers and administrators will be able to maintain the database and curated collections through their respective management tools.

### 5.1.4 Behavior and Interaction Flow

When the user enters the page, they are able to gain immediate access to various curated lists based on broad appeal (**FR-3.13.1**). If this is a user with an existing account who is signed in, these lists will instead be based upon the user's own interests.

When the user makes adjustments to the filters, the lists will be changed accordingly using the filters found within the database (**FR-3.13.4**). When the user sees a list that piques their interest, they can select the list, which will expand to show all of the resources found within that list.

Each resource will contain some information to give the user insight into what can be found. The user can then select a resource using the link provided within each item on the list, which will direct them to the external platform hosting that resource.

### 5.1.5 Test Plan

The following test plan will be used to verify that the **landing page** functions as intended. Testing will focus on the initial display of recommended resources, expanding resource information, and applying search filters. Each test identifies the user action, expected output, and actual output to determine whether the feature is functioning correctly. Any differences between the expected and actual results will be documented for further review.

| Test ID | Action | Input | Expected Output | Actual Output |
|---|---|---|---|---|
| landing_page_1 | Is opened | none | Page is automatically populated with recommended resources | |
| landing_page_2 | Opening a resource | Resource card is clicked on | The information relevant to the resource is | |

| | | | | |
|---|---|---|---|---|
| | | | expanded upon by a window sliding out from the right side that covers about 75% of the page | |
| landing_page_3 | Using filters | Clicking on filters found underneath the main search bar at the top of the page, such as “Computer Science” | The resources on the page will be filtered according to categories selected, in this case, the only resources that are viewable will be relevant to “Computer Science” | |
| landing_page_4 | Accessing additional filters | Filter button is clicked on (found hovering on the left side of the page) | Further filters, such as type, will be viewable. If search bar filters are changed from this menu, they will also be changed underneath the search bar | |

## 5.2 Resource Discovery with Filtering

When a user accesses the Resource Discovery page, the page will allow the user to search for resources and filter the results based on various criteria selected, showing only resources that are relevant to the user's needs (**FR-3.4.1**).

### 5.2.1 Components

Django is used for the frontend, database connection, and site presentation. MySQL is used for database management, while the Brave Search API is used to retrieve search results from the web. The Django backend handles communication between the database, search API, and frontend, including processing the selected search criteria and filtering returned resources.

### 5.2.2 Data

This page will read from the `category`, `filter`, `source`, and `price_model` tables, along with relevant information tied together through the `resource_category`, `resource_filter`, and `category_parent` junction tables. The `career_resource` table contains previously stored resources that may also be relevant to user searches.

The data retrieved from the search engine will be processed according to the search criteria and filters selected by the user. Source trust classifications will also be used to prevent resources from blocked sources from being displayed. Relevant resource information may be stored in the database for future use.

### 5.2.3 Interfaces

Before the user accesses this page, the software will communicate with the database to obtain a list of categories, filters, and other search options which the user can select during the process. The user interface will include a search bar at the top of the page, as well as some general filters underneath to help refine the search (**FR-3.4.2**).

Should the user desire further filtering, a small bubble on the left side can be selected to expand a left sidebar listing additional ways the search can be refined. After the search is completed, the resources will be listed in rows of cards with small summaries and links (**FR-3.4.3**).

As the user selects categories, filters, and other search options, these selections will be sent to the Django backend when applied. The backend will communicate with the Brave Search API to search the web for resources, process the returned results,

and display those matching the selected criteria (**FR-3.4.4**). When the user is satisfied with the results, they can select any of the resources listed.

### 5.2.4 Behavior and Interaction Flow

Upon user access of the Resource Discovery page, the user is prompted to make a search. The user can refine the search using the pre-generated filters found underneath the search bar, those accessible through the sidebar menu found upon clicking the bubble menu on the left side, or both.

After the search and filters are finalized by clicking on "Search" or "Apply Filters", the Django backend will process the search criteria and request results from the Brave Search API. The returned resources will be checked against the selected filters and source trust classifications before the page is populated with matching resources.

If the user finds a resource matching their needs, they can select the resource for further information or use the provided link to access it on the external platform.

### 5.2.5 Test Plan

The following test plan will be used to verify that Resource Discovery functions as intended. Testing will focus on search results, filtering accuracy, and handling searches with different levels of specificity. Tests will also verify that invalid or potentially harmful inputs do not affect the database or application functionality. Each test identifies the user action, expected output, and actual output to determine whether the feature is functioning correctly. Any differences between the expected and actual results will be documented for further review.

| Test ID | Action | Input | Expected Output | Actual Output |
|---|---|---|---|---|
| resource_discovery_1 | Searching for nothing | No inputs | Nothing will be populated, the user will be prompted to make a search | |
| resource_discorvery_2 | Broad search regarding resumes | “Resume” is typed into the search bar | Any resources that are populated will be pertaining to resumes, such as | |

| | | | | |
|---|---|---|---|---|
| | | | writing, formatting, and reviewing | |
| resource_discovery_3 | Narrowly targeted search for “Java” | “Java” is typed into search bar, filtered by “Computer Science”, “Free”, “Course” | Any resources shown should primarily have Java as the subject matter while also matching the filters selected. | |
| resource_discovery_4 | Sql injection | ; DROP tables resources; | The form does not recognize the command via input sanitization, so the user will unfortunately have this input as the search, which will populate the page with information regarding SQL commands | |

## 5.3 Onboarding: User Career and Skill Profiling

When a user signs up for CaLLP, they will be guided through an onboarding process to gather information about their fields of study, career interests, and skills. This information will be used to help personalize resource recommendations and future interactions with the website.

### 5.3.1 Components

Django is used for the frontend, database connection, and site presentation. MySQL is used for database management, while AWS is the hosting platform for the application and database. User authentication is planned through Cal Poly Humboldt SSO using Shibboleth, software that can implement SAML-based authentication.

The Django backend will handle processing the onboarding questionnaire, validating user input, and storing the information in the database (**FR-3.2.2**, **FR-3.2.3**, **FR-3.2.4**).

### 5.3.2 Data

The user can answer a questionnaire, which will populate the `user_interests` table. This table connects user ID numbers from the account table to various topics found in the category table, including skills, degree programs, and career fields. Users can also provide an interest score for skills.

The information collected will be used to make future curation and recommendations of resources easier for the user to access. The `account` table will store relevant user information, including their name, email, role, and account status.

### 5.3.3 Interfaces

The user will be able to answer the questionnaire directly on the webpage as part of the onboarding process. The interface will provide fields for account information and selections for fields of study, career interests, and skills (**FR-3.2.1**).

Once the user is finished, the information will be sent to the Django backend for validation and added to the database. This information can then be retrieved as needed for future use as the user continues to navigate through the website.

When university authentication is implemented, the application will communicate with Cal Poly Humboldt SSO to authenticate the user before completing the account creation process.

### 5.3.4 Behavior and Interaction Flow

When the user selects "Sign Up" by any means, they are taken to a separate page where they are prompted to provide information, such as name and email. When SSO is available, the user will be directed through the university's authentication process to verify their identity.

After this initial step, the user is directed to a questionnaire to help refine future interactions with the webpage, asking about topics such as fields of study, career interests, and skills. The user can select relevant categories and provide interest scores for skills where applicable.

Once all required fields have been filled, the Django backend will validate the information, create the account with a newly generated user ID, and store the questionnaire responses in the database. The user will then receive confirmation that their account has been created successfully.

### 5.3.5 Test Plan

The following test plan will be used to verify that the onboarding process functions as intended. Testing will focus on required fields, validation of user input, successful account creation, and storage of user interests in the database. Tests will also verify that invalid or potentially harmful inputs do not affect the database or application functionality. Each test identifies the user action, expected output, and actual output to determine whether the feature is functioning correctly. Any differences between the expected and actual results will be documented for further review.

| Test ID | Action | Input | Expected Output | Actual Output |
|---|---|---|---|---|
| onboarding_1 | Nothing | Nothing is answered, user proceeds to submit | The form is not submitted, user is prompted to complete all required fields | |
| onboarding_2 | Sql injection | ; DROP tables users; | The form does not recognize the command via input sanitization, so the user will unfortunately have this input as the relevant field, perhaps leading to termination of the account if discovered by administrator | |
| onboarding_3 | Various fields left blank | User does not input email or field of interest | The page will direct user to fill out the missing | |

| | | | | |
|---|---|---|---|---|
| | | | sections before being allowed to submit | |
| onboarding_4 | All fields are filled correctly | User fills out every section of the form correctly | The user will receive a confirmation message that the account is created; the information is updated to the database alongside a newly generated user id | |

# 6. Security and Privacy Design

This section describes the security and privacy measures used to protect the CaLLP application, user information, and external service integrations.

## 6.1 Application Authentication and Authorization

The CaLLP application plans to utilize Cal Poly Humboldt SSO through SAML 2.0 for user authentication. Django will manage user sessions and control access based on account roles and permissions. Administrative functions and protected user information will only be accessible to authorized accounts.

## 6.2 External Account Credential and Token Security

External learning and career platform integrations will utilize OAuth 2.0 with PKCE where supported. Access tokens and other credentials will be stored securely on the backend and will not be exposed to users. API keys, including the Brave Search API key, will be managed through environment variables. Token expiration, renewal, and revocation will be handled according to the requirements of each external service.

## 6.3 Data Protection and Privacy

The CaLLP application utilizes HTTPS to encrypt communication between users and the web server. AWS security configurations will be used to restrict database access and protect stored information. User data will only be collected as needed for application functionality, with access restricted according to account permissions. Data retention and deletion procedures will be established as development progresses.

## 6.4 Input Validation and Application Security

Django will be used to validate user input and protect against common web application threats, including SQL injection, cross-site scripting, and cross-site request forgery. Database operations will primarily use Django's ORM to reduce the risk of SQL injection. Additional validation will be applied to user-submitted forms, search inputs, and data received from external services.

# 7. Error Handling and Reliability

This section describes how the CaLLP application will handle errors, service failures, and data recovery to maintain reliable operation.

## 7.1 Error Handling Strategy

The CaLLP application will utilize Django's error handling capabilities to detect and manage application errors. Users will receive appropriate error messages when invalid information is submitted or an operation fails. Unexpected errors will be recorded for developers and administrators to review without exposing sensitive system information to users.

## 7.2 External Service Failure Handling

External services, including the Brave Search API and planned learning and career platform integrations, may experience outages, timeouts, or rate limits. The application will handle these failures by notifying users when a service is temporarily unavailable. Previously stored resources may remain accessible when external search services are unavailable.

## 7.3 Data Integrity and Recovery

The CaLLP database utilizes MySQL constraints, including primary keys, foreign keys, and validation checks, to maintain data integrity. Django database transactions will be used to prevent incomplete operations from leaving inconsistent data. AWS RDS backup and recovery capabilities will be used to support database restoration when needed.

## 7.4 Logging and Diagnostics

The CaLLP application will utilize Django and AWS logging capabilities to record application errors, failed requests, and other relevant system events. Logs will be used by developers and administrators for troubleshooting and maintenance. Sensitive information, including passwords, API keys, and authentication tokens, will not be recorded in application logs.

# 8. Design Rationale

This section describes the reasoning behind major technical decisions made during the development of the CaLLP application, along with alternatives considered and known limitations.

## 8.1 Major Design Decisions

The CaLLP application utilizes Django due to its built-in functionality for database management, user authentication, and web development. A modular monolithic architecture was selected to keep application components organized, avoid “spaghetti-code”, and simplify future development and maintenance.

MySQL was selected for relational database management, with AWS RDS providing database hosting and Elastic Beanstalk handling application deployment. AWS was chosen to provide a centralized hosting environment and simplify the eventual transfer of the application to its new owner.

The Brave Search API was selected for resource discovery, allowing CaLLP to locate external learning resources without hosting the content directly.

## 8.2 Alternatives Considered

Rather than developing the application without clearly defined module boundaries, the team selected a modular monolithic architecture to keep related functionality organized and reduce the risk of “spaghetti-code”. This approach allows the application to remain a single deployable system while maintaining separation between its major functional areas.

For application hosting, local deployment and the university's nrs-projects server were considered as alternatives to AWS. Both introduced complications with transferring ownership and supporting future development by other teams. AWS was selected for ease of deployment and ownership transfer, with MySQL chosen due to its availability through AWS's free-tier services.

## 8.3 Tradeoffs and Limitations

The modular monolithic architecture simplifies development and maintenance but requires the application to be deployed as a single unit. This may limit independent scaling of individual components.

The application's reliance on external services, including the Brave Search API and planned learning and career platform integrations, means some functionality will depend on third-party availability, rate limits, and API restrictions.

Hosting through AWS provides managed infrastructure but introduces ongoing service costs and dependence on AWS configurations. Resource discovery may also be limited by the availability and accuracy of information provided by external websites.

# Appendix A: Glossary

API – Application Program Interface
AWS – Amazon Web Services
CaLLP – Career & Lifelong Learning Pathway
ERD – Entity Relationship Diagram
HTTPS – HyperText Transfer Protocol Secure
JWT – JSON Web Token
MVT -Django's Model-View-Template
ORM – Object-Relational Mapping
PKCE – Proof Key for Code Exchange
RDS – Relational Database Service (via Amazon)
SAML – Security Assertion Markup Language
SDS – Software Design Specification
SQL – Structured Query Language
SRS – Software Requirements Specification
TLS – Transport Layer Security
WGAC – Web Content Accessibility Guidelines

# Appendix B: AI Assistance Disclosure

**AI Assistance Disclosure:** OpenAI's ChatGPT was used as a supplementary tool during preparation of this document to assist with brainstorming, terminology clarification, document organization, and drafting or refining selected language. The authors retain responsibility for requirements analysis, project-specific decisions, verification of content, and the final form of the document.